RISC-V ISA

Inhaltsverzeichnis

1 Ziel **1**

2 Instruktionen **3**
2.1 Konstanten (Immediate) . . . 4
2.2 Grundrechenarten . . . 4
2.3 Speicherzugriff . . . 4
2.4 Algorithmen . . . 4

3 Imperatives Programmieren **6**
3.1 Setup Ripes . . . 6
3.2 Branching . . . 7
3.3 Loops . . . 8
3.4 Functions . . . 8
3.5 Algorithms . . . 9
3.5.1 Fibonacci durch Rekursion . . . 9
3.5.2 Modulo . . . 10
3.5.3 Quadratzahl . . . 10
3.5.4 °F -> °C . . . 11

4 Assembler / Disassembler **13**
4.1 `HEIRV32-asm` Befehle . . . 13
4.1.1 Assembler zu Binär . . . 13
4.1.2 Binär zu Assembler . . . 14
4.1.3 Handgriff . . . 15
4.2 Main Funktion . . . 16
4.3 Reverse Engineering . . . 16

Akronyme **20**

1 Ziel

Dieser Labore ist in mehrere Blöcke unterteilt welche während verschiedenen Wochen durchgeführt wird. Das Ziel ist es mit sich mit der Assembler Sprache des RISC-V auseinanderzusetzten.

- Der Teil 2 Instruktionen des Labors betrachtet einzelne Instruktionen.
- Der Teil 3 Imperatives Programmieren des Labors führt uns zur imperative Programmierung

resp. Schleifen und Funktionen.

- Im letzten Teil 4 Assembler / Disassembler des Labors wird die Arbeit des Kompilers betrachtet : das Assembly und Disassembly eines Programms (Wechsel von High-Level-Code zu Maschinencode und umgekehrt).

2 Instruktionen

In diesem ersten Teil des Labors werden wir mit dem RISC-V Interpreter auf der Webseite https://hei-isc-car.github.io/riscv-interpreter/ arbeiten, siehe Abbildung 1. Dieser erlaubt es Register, Speicher zu beschreiben sowie den Code Schritt für Schritt auszuführen. Diese Onlinetools wird ihnen helfen Aufgaben zu lösen und zu kontrollieren.

Es dürfen nur Anweisungen verwendet werden, die auf dieser Seite unter **Supported Instructions** aufgelistet sind. Pseudo-Anweisungen werden nicht unterstützt.

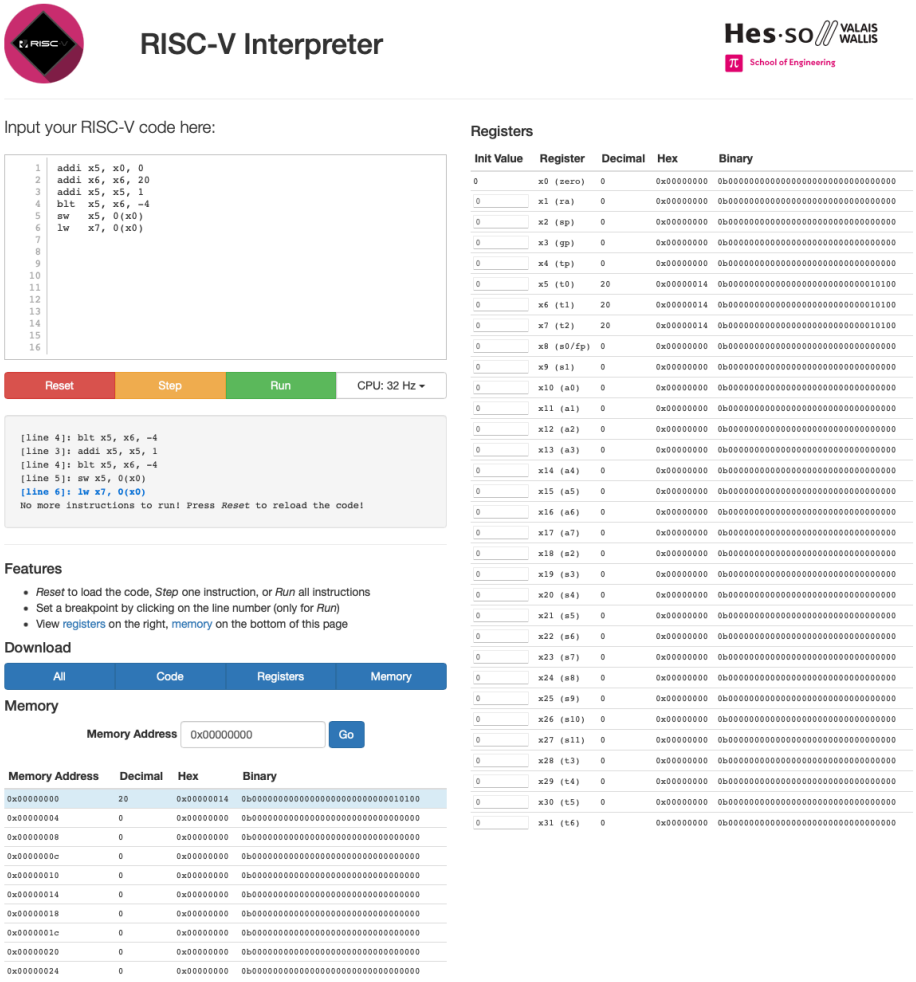


Abbildung 1: Online RISC-V Interpreter

Achten Sie sich auf die Typen der Variablen!

- Der Typ **int** wird als vorzeichenbehaftete 32-Bit-Größe betrachtet.
- Der Typ **unsigned int** wird als unsignierter 32-Bit-Typ betrachtet.
- Wenn eine Zahl dahinter steht (z. B. **int16_t**), bedeutet dies, dass die Variable x-Bit lang ist (hier 16). Wenn ein **u** vorangestellt ist, ist er unsigniert.

uint8_t ist also ein vorzeichenloses Byte, während **int8_t** ein vorzeichenbehaftetes Byte ist.

2.1 Konstanten (Immediate)

Schreibe den RV32i Assemblercode für folgende Instruktionen:

a)
```
int a = 10;
int b = 0;
a = a + 4;
b = a - 12;

int i = 0;
int x = 2032;
int y = -78;
```

b)
```
int a = 0xABCDE123;

int b = 0xFEEDA987;
```

2.2 Grundrechenarten

Schreibe den RV32i Assemblercode für folgende Instruktionen:

a)
```
int b = 1;
int c = 2;
a = b + c;

int b = -1;
int c = 2;
a = b + c;

int b = -12;
int c = 2023;
a = b + c;
```

b)
```
int b = 1;
int c = 2;
int d = 5;
a = b + c - d;

int b = -1;
int c = 2;
int d = -78;
a = b + c - d;

int b = -12;
int c = 2023;
int d = 22;
a = b + c - d;
```

c)
```
int b = 2;
int c = 3;
int e = -1;
int f = -78;
int g = 2023;
int h = -12;
a = b - c;
d = (e + f) - (g + h);
```

2.3 Speicherzugriff

Schreibe den RV32i Assemblercode für folgende Instruktionen:

a)
```
int a = mem[2];

int b = mem[8375];
```

b)
```
mem[5] = 42;
```

c)
```
uint16_t a = mem[3];
mem[4] = a;

int16_t a = mem[3];
mem[4] = a;
```

2.4 Algorithmen

Schreibe den RV32i Assemblercode für folgende Algorithmen, benutzte nur die Grundinstruktionen ohne *loops conditionals* sowie *branches*:

a) In einem System kann es notwendig sein, abzuwarten, ohne etwas zu tun, ohne den aktuellen Zustand des Systems zu verändern (kein Speicherzugriff, keine Registeränderung). Diese Operation wird gemeinhin als **NOP** (**NO O**peration) bezeichnet. Schlagen Sie eine Anweisung dafür vor.

b) Berechne die ersten 10 Fibonacci Nummern. Speichern Sie jede Zahl in einem anderen Register.

c) Übertrage den 8Bit Wert des Speichers an Addresse $0x0000'1000$ seriell Bit für Bit im LSB des Speichers in der Addresse $0x0000'1001$. Die restlichen Bits der Speicheraddresse $0x0000'1001$ müssen '0' betragen. Berechnen sie die BaudRate in $\frac{Instructions}{Bit}$ für die gesamte Übertragung?

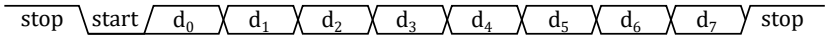


Abbildung 2: UART Serielle Übertragung

d) Multipliziere zwei 4-bit Zahlen zusammen benutzte hierzu zusätzlich einen der Befehle bne, bge. Der Algorithmus funktioniert folgendermassen: Eine Multiplikation ist das gleiche wie die x-fache Addition der gleichen Zahl. Zum Beispiel: $2 * 9 = 9 + 9 = 18$.

3 Imperatives Programmieren

In diesem Labor können Sie weiter den Online RISC-V Interpreter benutzten. Desweiteren steht Ihnen auch der Program **Ripes** zu Verfügung, siehe Abbildung 3 Ripes Grafische Entwickleroberfläche. Dieses müssen Sie zuerst herunterladen und konfigurieren.

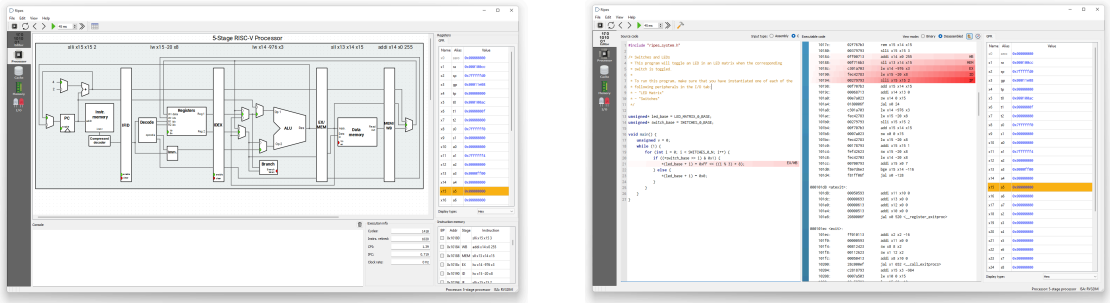


Abbildung 3: Ripes Grafische Entwickleroberfläche

3.1 Setup Ripes

> Auf Labor-PCs befindet sich unter **C:/eda/RiscV** Ripes und der unten beschriebene **gcc** Compiler.

1. Laden Sie die letzte Release Version des Programmes für Ihre Platform herunter unter dem Link: https://github.com/mortbopet/Ripes/releases.
2. Laden Sie die folgende Version der RISC-V-GNU-Toolchain für Ihre Platform unter folgendem Link herunter: https://github.com/sifive/freedom-tools/releases/tag/v2020.04.0-Toolchain.Only
3. Entpacken Sie die beide und kopieren Sie den RISC-V-GNU-Toolchain Ordner in den Ripes Ordner.
4. Starten Sie Ripes und konfigurieren Sie die Kompiler Einstellungen unter: *Edit* → *Settings*. Hierzu müssen sie die Datei `bin/riscv64-unknown-elf-gcc-8.3.0.exe` auswählen.

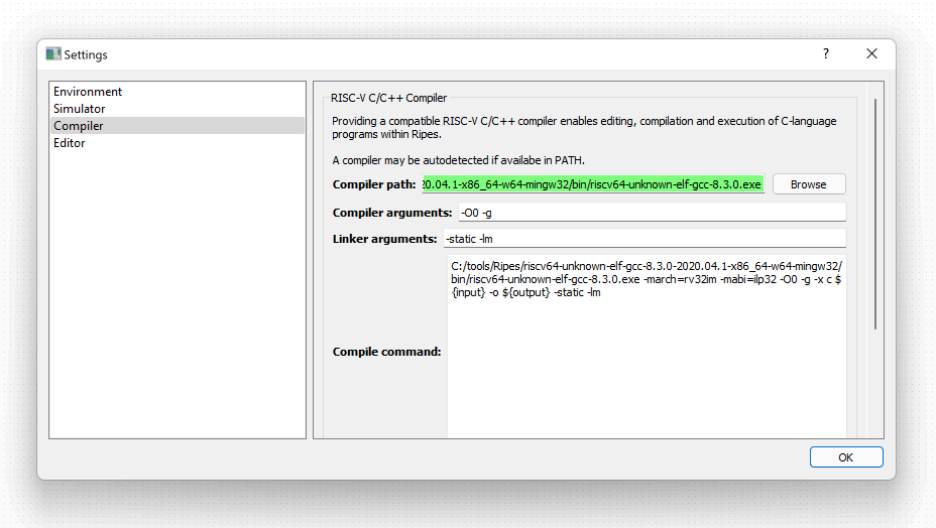


Abbildung 4: Ripes Toolchain Einstellungen

5. Wählen Sie in den Prozessor Einstellungen den RISC-V → 32-bit → Single-Cycle Processor ohne ISA Extentions aus.

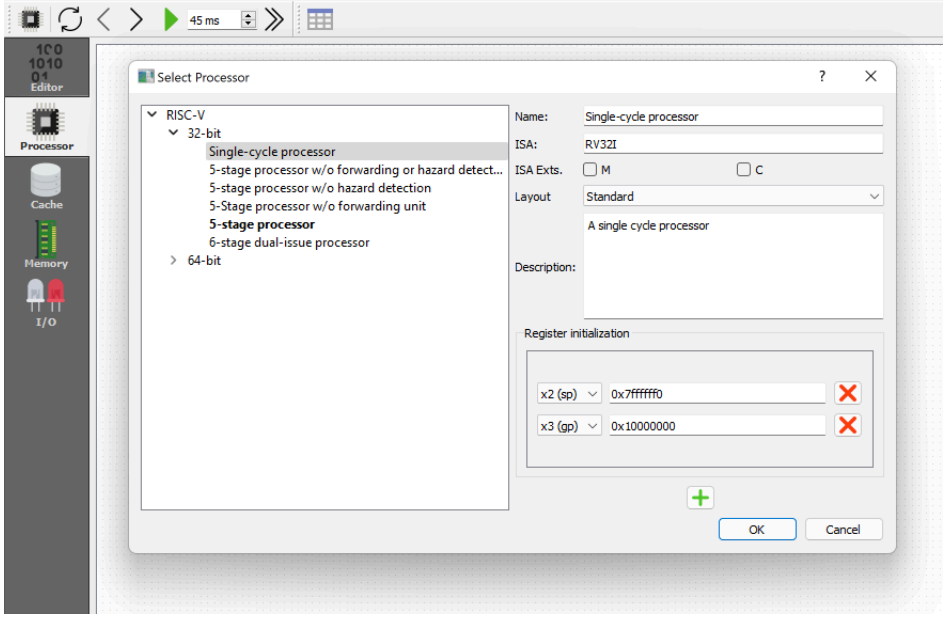


Abbildung 5: Ripes Prozessor Einstellungen

3.2 Branching

If / else

a)
```
int a = 1, b = 2, c;

if(a == b) {
  c = 1;
}
else {
  c = 0;
}
```

b)
```
int a = 1, b = 2, c;

if(a == b) {
  c = 0;
} else if(a > b) {
  c = 1;
} else {
  c = 2;
}
```

Switch case

a)
```
int a, b;

switch(b) {
  case 0:
    a = 17;
    break;
  default:
    a = 99;
}
```

b)
```
int a;

switch(mem[2]) {
  case 0:
    a = 17;
    break;
  case 3:
    a = 33;
    break;
  case 8:
  case 12:
    a = 10;
    break;
  default:
    a = 99;
}
```

3.3 Loops

While / Do While

a)
```
// A
int a = 10;

do{a = a - 1;}
while(a != 0);

// B
int a = 10;

while(a >= 0)
{a = a - 1;}
```

b)
```
// A
int a = 10;

do{a = a - 1;}
while(a != 0);

// B
int a = 10;

do{a = a - 1;}
while(a >= 0);

// C
unsigned int a = 10;

do{a = a - 1;}
while(a >= 0);
```

For

a)
```
unsigned int a = 0, i;

for(i = 0; i < mem[0]; i = i + 1) {
  a = a + 2;
}
```

b)
```
int a = 0, i;

for(i = 4; i > mem[0]; i = i - 1) {
  a = a + i;
}
```

c)
```
// An array of 10 bytes
uint8_t myArray[10] = ...
// ...

// Let say myArray[0] is at the
// address saved in register s0.
// Arrays in C are contiguous in memory.

int i;

for(i = 0; i < 10; i = i + 1) {
  myArray[i] = myArray[i] - 5;
}
```

3.4 Functions

Obwohl ein einzelner Programmierer seinen Code nach Belieben anordnen kann, wurden bestimmte Konventionen festgelegt, um die Interoperabilität verschiedener Codequellen untereinander zu ermöglichen.

Die Funktion die eine andere aufruft heisst **caller** und die aufgerufene Funktion heisst **callee**. Sie müssen synchronisiert sein, dies Bedeutet wie Argumente übergeben werden, welche Register beibehalten werden müssen ...

Die folgenden Konzepte sind die wichtigsten Regeln:

- In den Registern **a0** bis **a7** können Argumente übergeben werden. Wenn mehr benötigt werden, werden sie auf dem Stack übergeben.
- Das Ergebnis wird im Register **a0** zurückgegeben.
- Der **caller** speichert die durch die Anweisung **jal** erzeugte Rücksendeadresse (PC + 4) im Register **ra**.
- Der **callee** darf die Rücksprungadresse, den Stack oder die gespeicherten Register sXX nicht neu schreiben. Falls sie benutzt werden, muss der Platz auf dem Stack reserviert und diese

Informationen gespeichert und anschliessend wiederhergestellt werden.

a)
```
doNothing();

...

void doNothing() {
  return;
}
```

b)
```
int a = 1, b;
b = doubleIt(a);
b = doubleItOpti(a);

...

// Non-optimized version
// Let's assume a is saved in s0
int doubleIt(int myvar) {
  int a = myvar; // we WANT a in s0 !
  a = a * 2;
  return a;
}

// Optimized version
// Choose your registers freely
// Try having the less possible
//   instructions
int doubleItOpti(int myvar) {
  int a = myvar; //
  a = a * 2;
  return a;
}
```

c)
```
int a = 1, b;
b = callA(a);

...

// Functions can be
// optimized at will

int callA(int v1) {
  v1 = v1 * 2;
  return callB(v1);
}

int callB(int v1) {
  v1 = v1 + 12;
  return v1;
}
```

d)
```
int a=1, b=2, c=3,
 d=4, e=5, f=6, g=7,
 h=8, i=9, j=10, res;
res = sum(a,b,c,d,e,f,
  g,h,i,j);

...

int sum(int v1, int v2,
  int v3, int v4, int v5,
  int v6, int v7, int v8,
  int v9, int v10){
  int c;
  c = v1 + v2 + v3 + v4 +
    v5 + v6 + v7 + v8 +
    + v9 + v10;
  return c;
}
```

3.5 Algorithms

3.5.1 Fibonacci durch Rekursion

Der Begriff Rekursivität impliziert, dass eine Funktion sich selbst aufruft, um ein Ergebnis zu berechnen, so wie sie auch eine andere Funktion aufrufen würde.

Das Ziel ist es, die Fibonacci-Folge einer unsignierten ganzen Zahl nach dem folgenden Algorithmus zu berechnen:

```
unsigned int fibbonacci(unsigned int n){
  if(n == 0){
    return 0;
  } else if(n == 1) {
    return 1;
  } else {
    return (fibbonacci(n-1) + fibbonacci(n-2));
  }
}
```

- Implementieren Sie die gegebene Funktion
- Testen und Validieren der Funktion
- Welche n-Werte werden unterstützt?

Es ist möglich, die Berechnung stark zu beschleunigen, indem man die Speicherung verwendet. Das Prinzip besteht darin, Speicherplatz zu reservieren und die bereits bekannten Ergebnisse für jedes berechnete n zu speichern und diesen Wert direkt wieder zu verwenden. RV32IM wird hier empfohlen.

- Implementieren Sie die gegebene Funktion
- Testen und Validieren der Funktion
- Vergleichen Sie die Algorithmen.

3.5.2 Modulo

Der Modulo **%** ist eine Operation, die auf zwei positive ganze Zahlen angewendet wird und ist nichts anderes als der Rest der Division. Zum Beispiel ergibt 5 geteilt durch 3 ergibt 1 (man kann 3 in 5 einmal hineinlegen), Rest 2.

Das Modulo einer Zahl durch 0 ist nicht definiert.

Die Definition für vorzeichenbehaftete Zahlen weicht je nach Sprache ab. Wir behandeln hier nur die unsigned Version.

Modulo ist vielseitig einsetzbar und ermöglicht es, Werte zu begrenzen, Informationen zu extrahieren, eine X- und Y-Position aus einem X*Y-Wert zu berechnen in einem Array mit bekannter Grösse …

- Geben Sie einen Code an, mit dem diese Operation für jede positive ganze Zahl unter Verwendung des Sets RV32IM durchgeführt werden kann.
- Wie kann das Gleiche in RV32I implementiert werden? Beschreiben Sie das/die Konzept(e).

Die Aufgabe des Compilers ist es, den Code so gut wie möglich zu optimieren. Wenn die erkannte Operation ein Modulo ist, bei dem eine Konstante eine Potenz von 2 ist (z. B. x % 2, y % 8 …), ist eine Variante möglich, die keine Division enthält.

- Geben Sie diese Variante an.

> Das Konzept des Modulo für reelle Zahlen kam mit der Entwicklung der Rechenleistung auf und das Ergebnis weicht auch je nach Sprache ab. In C ist eine spezielle Funktion der std-Bibliotheken erforderlich, **fmod()**. In Python ist diese Operation nativ. In jedem Fall sind sie ressourcenintensiver und erfordern die Behandlung einiger Sonderfälle (NaN, ∞).

3.5.3 Quadratzahl

Eine quadrierte Zahl ist nichts anderes als die Multiplikation der genannten Zahl mit sich selbst. Hier arbeiten wir nur mit dem Befehlssatz RV32I:

- Schlagen Sie eine Funktion vor, die $n * n$ durch eine Additionsschleife berechnen kann.
- Testen Sie mit n = 5, 10 und 20.
- Verallgemeinern Sie die Funktion so, dass Sie ihr zwei Parameter liefern können, indem

Sie $i * j$ berechnen.

- Testen Sie mit i = 5, j = 100. Testen Sie mit i = 100, j = 5. Was stellen Sie fest?
- Optimieren Sie die Funktion so, dass die Reihenfolge der Operanden keinen Einfluss auf die Rechenzeit hat.

3.5.4 °F -> °C

Wir möchten eine Funktion erstellen, die Celsiusgrade in Fahrenheit umwandeln kann, da die Fahrenheitswerte zwischen 32 und 1000 schwanken, ist eine Genauigkeit auf das Grad ausreichend.

Die Formel ist einfach: $C = (F - 32) * \frac{5}{9}$.

Es wäre praktisch wenn eine FPU zur Verfügung stünde, aber es wird nur den RV32I-Basisbefehlssatz unterstützt.

Um dieses Problem zu umgehen, können Sie einige Tricks anwenden, der Algorithmus lautet wiefolgt:

- Berechnen Sie $C = F - 32$
- Mit 5 multiplizieren
- Durch 9 teilen
  - $\frac{1}{9}$ kann durch eine spezielle Binäre Darstellung gespeichert werden

    ```
    +-----+-----+-----+-----+-----+-----+------------+------------+
    | b31 | b30 | b29 | b28 | b27 | ... |         b1 |         b0 |
    +-----+-----+-----+-----+-----+-----+------------+------------+
    | 2^0 | 2^-1| 2^-2| 2^-3| 2^-4| ... |      2^-30 |      2^-31 |
    +-----+-----+-----+-----+-----+-----+------------+------------+
    | 1   | 1/2 | 1/4 | 1/8 | 1/16| ... |1/1737418240|1/2147483648|
    +-----+-----+-----+-----+-----+-----+------------+------------+
    ```

    ins diesem Fall ist dies $0000_1110_0011_1000_1110_0011_1000_1110_2$.
  - Die Konstante kann vorberechnet werden und beträgt $\frac{2^n}{9} + 1$. Je grösser n ist, desto höher ist die Genauigkeit. Die Anzahl Bits definieren die maximale Grösse von n. Die $+1$ ist eine Aufrundung für die verlorene Genauigkeit.
  - Nehmen wir $n = 16$. Unsere magische Zahl lautet daher $magic = \frac{2^n}{9} + 1 = \frac{65536}{9} + 1 = 7282$.
  - Wir müssen den Wert mit dieser magischen Zahl multiplizieren und dann durch 2^ndividieren. Im Fall $n = 16 \rightarrow \frac{1}{65536}$

Um die Arbeit zu vereinfachen, werden mehrere Annahmen getroffen:

- Die magische Zahl und die Temperatur in Fahrenheit sind immer positiv.
- Die Grösse der grössten Multiplikation beträgt:

$$\begin{aligned} nbBits_{max_{fahrenheit}} + nbBits_{mult5} + nbBits_{magicNumber} &= \\ 10\textit{(max. 1000-32)} + 3 + (n - nbBits_{div9} + 1) &= \\ 10 + 3 + (16 - 4 + 1) &= \\ &= 26bits \end{aligned}$$

Sie überschreitet nie 32 Bit (für $n < 23$). chreiben Sie den entsprechenden Code.

Testen und optimieren Sie die Funktion:

- Testen Sie mit verschiedenen Werten von Fahrenheit.
- Testen mit mehreren Werten für n (16, 18, 20). *Vergessen Sie nicht die magische Zahl neu zu berechnen.*
- Testen Sie die Funktion mit $n = [22, 23]$, für $F = [100, 400, 1000]$.

4 Assembler / Disassembler

Den RISC-V Assembler und Disassembler welcher im Labo benutzt wird kann im Repository unter `heirv32_sc/heirv32-asm/` für Windows & Linux amd64 sowie macOS ARM64 gefunden werden. Diese Tool erlaubt es ihnen RV32i oder HEIRV32 Assemblercode in Binärcode zu verwandeln (Assembly) sowie Binärcode zurück in Assemblercode zu verwandeln (Disassembly).

```
$ heirv32-asm --help

usage: HEIRV32-ASM
    Assembler/Disassembler to support HEIRV32 ISA,
    See help below for usage : it auto-detects if the file is binary like (=> will disassemble)
    or contains ASM instructions (=> will assemble). If no switches are given, will open a GUI
    to select the file and run with '-t = 'phb'' argument.

    Usage:
    heirv32 -f <ASMfile> [-t="phb"] [-i="HEIRV32" | "RV32I"]
    heirv32 -s <string> [(-t="hb" -of <outputPathName>)] [-i="HEIRV32" | "RV32I"]
    heirv32 -g [-t="phb"] [-i="HEIRV32" | "RV32I"]
    heirv32 -i <ISA>
    heirv32 -h | --help
    heirv32 [-g -t="phb" -i="HEIRV32"]

    Options:
    -h --help     Show this screen.
    -f            Input ASM file.
    -s            Input string.
    -g            Open GUI to select file to convert.
    -i            ISA to use ('HEIRV32' or 'RV32I' supported), default 'HEIRV32'.
                  Can be used alone to log the ISA specs.

    -t            Output type ('p' - print, 'h' - BRAM file, 'b' - bin File), default 'p'.
    -of           Output file path whe convertin an input string ('-s'), if 'h' and/or 'b'
                  are used for '-t'.
```

4.1 `HEIRV32-asm` Befehle

Abhänging vom Eingangsfiles wird automatisch assembliert oder disassembliert

4.1.1 Assembler zu Binär

```
$HEIRV32-ASM_1.1.1_Darwin_ARM -f ./tests/tassembly.s -t p

Generating file with output type p

File: ./tests/tassembly.s
Conversion ongoing
 * Using the ASSEMBLER
 * Found a value in code : 1500 at address 88 - do not let your code try to run it as
 ↪  an instruction !
 * Found a value in code : 1 at address 92 - do not let your code try to run it as an
 ↪  instruction !
 * Found a value in code : 255 at address 96 - do not let your code try to run it as
 ↪  an instruction !
 ** Found data / instructions: 25

------ p : Printing instructions list and write to file ------
 * Output file: _syn.txt
0x0000: addi x2 x0 5      => 0b00000000'01010000'00000001'00010011 => 0x00500113
```

```
0x0004: addi x3 x0 12     => 0b00000000'11000000'00000001'10010011 => 0x00c00193
0x0008: addi x7 x3 -9     => 0b11111111'01110001'10000011'10010011 => 0xff718393
0x000c: or   x4 x7 x2     => 0b00000000'00100011'11100010'00110011 => 0x0023e233
0x0010: and  x5 x3 x4     => 0b00000000'01000001'11110010'10110011 => 0x0041f2b3
0x0014: add  x5 x5 x4     => 0b00000000'01000010'10000010'10110011 => 0x004282b3
0x0018: beq  x5 x7 end    => 0b00000010'01110010'10001000'01100011 => 0x02728863
0x001c: slt  x4 x3 x4     => 0b00000000'01000001'10100010'00110011 => 0x0041a233
0x0020: beq  x4 x0 around => 0b00000000'00000010'00000100'01100011 => 0x00020463
0x0024: addi x5 x0 0      => 0b00000000'00000000'00000010'10010011 => 0x00000293
0x0028: slt  x4 x7 x2     => 0b00000000'00100011'10100010'00110011 => 0x0023a233
0x002c: add  x7 x4 x5     => 0b00000000'01010010'00000011'10110011 => 0x005203b3
0x0030: sub  x7 x7 x2     => 0b01000000'00100011'10000011'10110011 => 0x402383b3
0x0034: sw   x7 84(x3)    => 0b00000100'01110001'10101010'00100011 => 0x0471aa23
0x0038: lw   x2 88(x0)    => 0b00000101'10000000'00100001'00000011 => 0x05802103
0x003c: add  x9 x2 x5     => 0b00000000'01010001'00000100'10110011 => 0x005104b3
0x0040: jal  x3 end       => 0b00000000'10000000'00000001'11101111 => 0x008001ef
0x0044: addi x2 x0 1      => 0b00000000'00010000'00000001'00010011 => 0x00100113
0x0048: add  x2 x2 x9     => 0b00000000'10010001'00000001'00110011 => 0x00910133
0x004c: sw   x2 0x20(x3)  => 0b00000010'00100001'10100000'00100011 => 0x0221a023
0x0050: beq  x2 x2 main   => 0b11111010'00100001'00001000'11100011 => 0xfa2108e3
0x0054: jal  x3 main      => 0b11111010'11011111'11110001'11101111 => 0xfadff1ef
0x0058: 1500              => 0b00000000'00000000'00000101'11011100 => 0x000005dc
0x005c: 0b1               => 0b00000000'00000000'00000000'00000001 => 0x00000001
0x0060: 0xff              => 0b00000000'00000000'00000000'11111111 => 0x000000ff
```

4.1.2 Binär zu Assembler

```
./dist/HEIRV32-ASM_1.1.1_Darwin_ARM -f ./tests/tassembly.txt -t p

Generating file with output type p

File: ./tests/tassembly.txt
Conversion ongoing
 * Using the DISASSEMBLER
 * Finding instructions
  ** 00000000000000000000010111011100 no match - assuming value
  ** 00000000000000000000000000000001 no match - assuming value
  ** 00000000000000000000000011111111 no match - assuming value
 * Recomposing rd, rs and immediates
 * Adding labels
['addi x2 x0 5', 'addi x3 x0 12', 'addi x7 x3 -9', 'or x4 x7 x2', 'and x5 x3 x4', 'add
↪  x5 x5 x4', 'beq x5 x7 48', 'slt x4 x3 x4', 'beq x4 x0 8', 'addi x5 x0 0', 'slt x4
↪  x7 x2', 'add x7 x4 x5', 'sub x7 x7 x2', 'sw x7 84(x3)', 'lw x2 88(x0)', 'add x9 x2
↪  x5', 'jal x3 8', 'addi x2 x0 1', 'add x2 x2 x9', 'sw x2 32(x3)', 'beq x2 x2 -80',
↪  'jal x3 -84', '(value) 1500', '(value) 1', '(value) 255']
 Modifying beq x2 x2 -80 with label 0 - instr nb. 20
 Modifying jal x3 -84 with label 0 - instr nb. 21
 Modifying beq x4 x0 8 with label 1 - instr nb. 8
 Modifying beq x5 x7 48 with label 2 - instr nb. 6
 Modifying jal x3 8 with label 2 - instr nb. 16
 * Recomposed labels / instructions: 28

------ p : Printing instructions list ------
lb0:
addi x2 x0 5
addi x3 x0 12
addi x7 x3 -9
or x4 x7 x2
and x5 x3 x4
add x5 x5 x4
beq x5 x7 lb2
slt x4 x3 x4
beq x4 x0 lb1
addi x5 x0 0
```

```
lb1:
slt x4 x7 x2
add x7 x4 x5
sub x7 x7 x2
sw x7 84(x3)
lw x2 88(x0)
add x9 x2 x5
jal x3 lb2
addi x2 x0 1

lb2:
add x2 x2 x9
sw x2 32(x3)
beq x2 x2 lb0
jal x3 lb0
(value) 1500
(value) 1
(value) 255
```

4.1.3 Handgriff

- Speichern Sie den folgenden Code als **.s**-Datei:

```
myLabel:
  li   x20 0b11111111111
  add  x20 x20 x20
  add  x20 x20 x20
  add  x20 x20 x20
  addi x3 x0 0
begin:
  mv   x1 x0
a: # Should be useful for something
  slt  x2 x1 x20
  beq  x2 x0 subing
  addi x1 x1 1
  beq  x0 x0 a
subing:
  addi x3 x3 -1
  beq  x3 x0 test
label_wo_beq:
  addi x30 x0 0xCC
  jal  ra begin
test:
  addi x30 x0 0b00110011
  jal  begin
l6: # This is very important
# But won't always save you
  jal  ra myLabel
```

- Kompilieren Sie es mit HEIRV32-ASM, wobei die Ausgaben **print** und **bin** aktiviert sind:
  `./dist/HEIRV32-ASM_1.1.1_xxx -i RV32I -f ./my/file/disassembly-01.s -t pb`.
  Eine Datei **.txt** wird erstellt, die eine für Menschen lesbare hexadezimale Version der erzeugten Datei **.bin** ist.
- Nun dekompilieren Sie die Datei **.txt** auf die gleiche Weise, um eine Assemblerdatei neu zu erstellen:
  `./dist/HEIRV32-ASM_1.1.1_xxx -i RV32I -f ./my/file/assembled-01.txt -t pb`.
  Dies sollte eine Datei ***_disassembly.s** erstellen.
- Was ändert sich zwischen den beiden Codes ?
- Wozu dient er übrigens?

4.2 Main Funktion

Im Program Ripes gezeigt unter 3.1, Kompilieren Sie die Folgenden Codes und suchen Sie im Assemblercode die entsprechenden Assemberbefehle. Was bedeuten die einzelnen Befehle im `main:` ?

```
void main() {
  int a = 0;
}
```

```
void main() {
  while(1) {
    int a = 0;
  }
}
```

4.3 Reverse Engineering

Sie arbeiten in einem Partnerprojekt an einem etwas modifizierten RISC-V-Prozessor.

Um sich das Leben zu vereinfachen und mit der Aussenwelt zu kommunizieren, es kann den Status von 4 Tasten die mit dem Prozessor verbundenen auslesen, indem er die Speicheradresse **0xf0000000** liest, und 8 LEDs einschalten, indem er an die Speicheradressen **0xf0000004** (led 0) bis **0xf0000020**(led 7) schreibt:

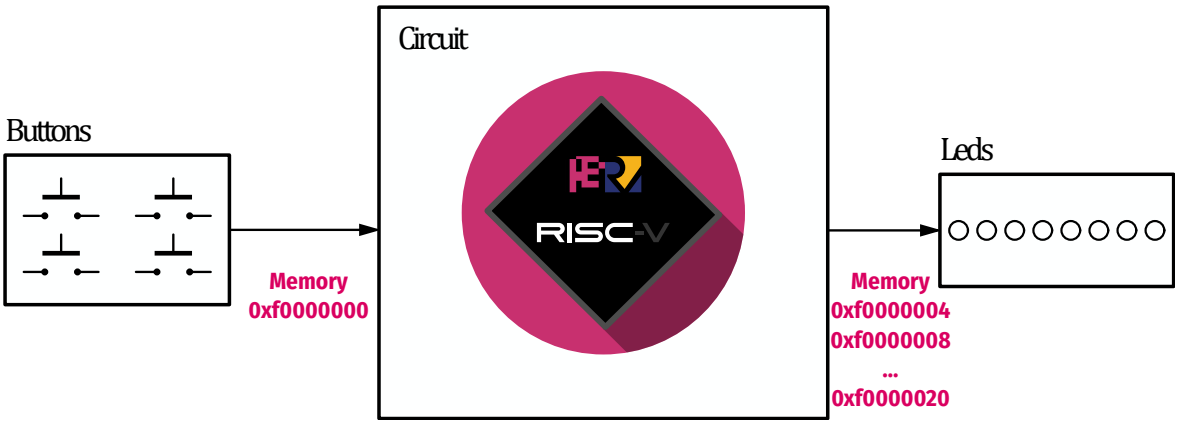


Abbildung 6: Modifizierter RISC-V-Prozessor

Das erste Labor bestand darin, die Leds entsprechend der gedrückten Tasten blinken zu lassen:

- Keine Taste: Die 8 LEDs werden nach dem Muster 0xAA eingeschaltet
- Der Knopf 0 ist auf '1': Alle 8 LEDs blinken in einer angemessenen Frequenz (das Blinken ist sichtbar)

Um den Zugriff auf die Taste und die Leds zu vereinfachen wurde Ihnen der folgende Code gegeben:

```
setup:
    # led is sw xx, offBy4(x30) with xx loaded as 0x00rrggbb
    li x30, 0xf0000004 # base address for leds
    li x31, 0xf0000000 # base address for buttons, one register

    # DO NOT MODIFY x30 (t5) and X31 (t6) !!!

get_btns: # return buttons value in a0
  lw a0, 0(x31)
```

```
  jr ra

set_leds: # pass a 8 bits which if bit(n) = '1', led(n) is on
  li t0, 8   # loop the leds
  mv t3, x30 # mem position
  addi t3, t3, 28 # leds display is reversed, so stock reverted

  leds_loop:
  beq x0, t0, leds_end
  andi t1, a0, 1
  # if 1, lights corresponding led
  beq t1, x0, isoff

  ison:
  li t2, 0x00ff00ff
  j leds_loop_end

  isoff:
  mv t2, x0

  leds_loop_end:
  sw t2, 0(t3) # save led value
  srli a0, a0, 1  # shift right leds value
  addi t0, t0, -1 # decrement loop
  addi t3, t3, -4  # add memory pos
  j leds_loop

  leds_end:
  jr ra
```

Leider ist Ihr Kollege heute abwesend und hat Ihnen keine Kopie des bereits geschriebenen Codes hinterlassen. Ausserdem gab es einige Bugs:

- Keine Tasten: die Leds leuchten nach dem Muster 0x2A auf
- Der Knopf 0 ist auf '1': nur 7 LEDs blinken, und das mit einer Frequenz, die viel zu hoch zu sein scheint.
- Eine der anderen Schaltflächen wird gedrückt: Dasselbe Verhalten, als wenn die Schaltfläche 0 gedrückt wird

Es scheint, dass Sie die Schleife, in der Sie auf das Blinken warten, falsch berechnet verwendet haben. Ausserdem folgen die Leds nicht den angegebenen Mustern und mit einer zu hohen Geschwindigkeit. Ein Hardwareproblem kann ausgeschlossen werden.

Sie erinnern sich, dass der zuletzt geschriebene Code auf Ihr Board RISC-V geflasht wurde, und da der Chip nicht gegen Rücklesen geschützt war, können Sie den Code durch Zugriff auf den JTAG-Bus extrahieren:

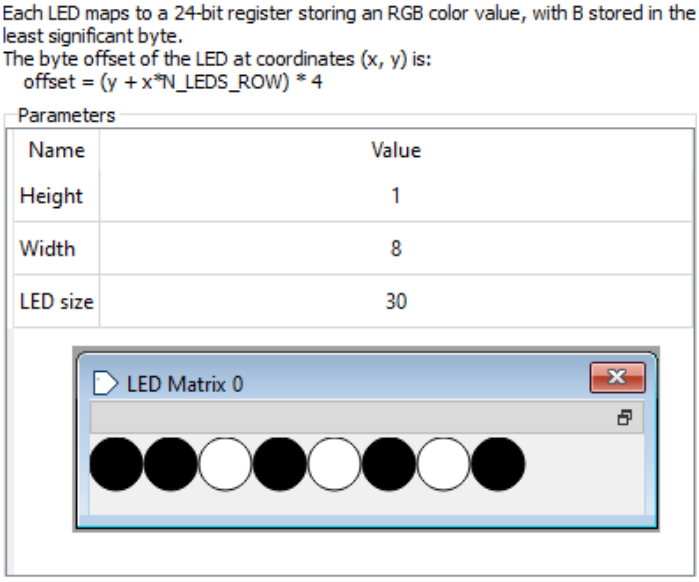
```
f0000f37
00400f13
f0000fb7
00000f93
00000413
040000ef
02a00463
00900663
fff48493
ff1ff06f
06400493
00800663
00000413
0140006f
0bf00413
```

```
00c0006f
02a00413
00000493
00040513
010000ef
fc5ff06f
000fa503
00008067
00800293
000f0e13
01ce0e13
02500663
00157313
00030663
00fff3b7
fff00393
0080006f
00000393
007e2023
00155513
fff28293
ffce0e13
fd5ff06f
00008067
```

- Extrahieren Sie Ihren Code
- Identifizieren und trennen Sie den gegebenen Code vom Rest
- Die Verwendung von gegebenen Funktionen verstehen

Um das System zu simulieren, öffnen Sie unter Ripes die Registerkarte I/O:

- Doppelklicken Sie auf LED Matrix, wählen Sie das erstellte Widget aus und konfigurieren Sie es so, wie es ist:



Exports

```
#define LED_MATRIX_0_BASE (0xf0000004)
#define LED_MATRIX_0_SIZE (0x20)
#define LED_MATRIX_0_WIDTH (0x8)
#define LED_MATRIX_0_HEIGHT (0x1)
```

Abbildung 7: Ripes - Led-Konfiguration

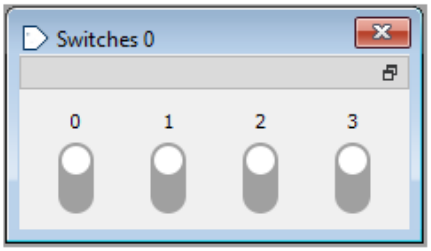
Stellen Sie sicher, dass **Exports** die gleichen Informationen enthält wie auf dem Bild, das nach der Einstellung des Moduls gegeben wurde.

- Doppelklicken Sie auf Switches, wählen Sie das erstellte Widget aus und konfigurieren Sie es so, wie es ist:

Each switch maps to a bit in the memory-mapped register of the peripheral.
switch n = bit n

Parameters

Name	Value
# Switches	4



Exports

```
#define SWITCHES_0_BASE (0xf0000000)
#define SWITCHES_0_SIZE (0x4)
#define SWITCHES_0_N (0x4)
```

Abbildung 8: Ripes - Schalterkonfiguration

Stellen Sie sicher, dass **Exports** die gleichen Informationen enthält wie auf dem Bild, das nach der Einstellung des Moduls gegeben wurde.

Laden Sie Ihren Assemblercode in **Editor**, drücken Sie F8 (oder das Symbol ») und stellen Sie fest, dass der Schaltkreis gemäss den genannten Problemen funktioniert, indem Sie unter **I/O** mit den Schaltern interagieren.

- Fixieren Sie die genannten Probleme
- Testen Sie Ihre Rennstrecke auf Ripes

Da die Clock nicht genau eingestellt werden kann, wird die Blinkfrequenz der Leds ohne Berechnung ermittelt. Korrigieren Sie den Code, um etwa 2 Hz zu erhalten.

Was Sie gerade getan haben, ähnelt dem Hardware-Hacking: Ein Code wird gedumpt (kopiert), analysiert, verändert und dann wieder in ein System eingespeist, um es nach Lust und Laune anzupassen.
Natürlich ist es nicht immer so einfach, einen Code zu kopieren und neu einzuspeisen. Es sind teilweise exotische Methoden entstanden (siehe den Kamikaze-Hack der Xbox 360).

Akronyme

FPU **F**loating **P**oint **U**nit. 10

ISA **I**nstruction **S**et **A**rchitecture. 1

LSB **L**east **S**ignifiant **B**it. 4

RISC-V **R**educed **i**nstruction **s**et **c**omputer **V**. 1, 2, 5, 12, 15, 16